

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph and Tree

Oleh:

Hafizh Pratama Budiman

NIM. 2210817310007

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

BULAN JUNI

TAHUN 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph and Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Hafizh Pratama Budiman
NIM : 2210817310007

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI.....	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL.....	5
SOAL 1.....	6
A. Source Code.....	7
B. Output Program.....	10
C. Pembahasan.....	10
SOAL 2.....	12
A. Source Code.....	13
B. Output Program.....	15
C. Pembahasan.....	15
SOAL 3.....	18
A. Source Code.....	20
B. Output Program.....	22
C. Pembahasan.....	22
SOAL 4.....	24
A. Source Code.....	26
B. Output Program.....	28
C. Pembahasan.....	28
SOAL 5.....	31
A. Source Code.....	33
B. Output Program.....	36
C. Pembahasan.....	36
TAUTAN GIT.....	39

DAFTAR GAMBAR

Gambar 1 Output Dari Soal 1.....	10
Gambar 2 Output Dari Soal 2.....	15
Gambar 3 Output Dari Soal 3.....	22
Gambar 4 Output Dari Soal 4.....	28
Gambar 5 Output Dari Soal 5.....	36

DAFTAR TABEL

Tabel 1 Contoh Kasus Soal 1.....	7
Tabel 2 Source Code soal1.cpp.....	7
Tabel 3 Contoh Kasus Soal 2.....	13
Tabel 4 Source Code soal2.cpp.....	13
Tabel 5 Contoh Kasus Soal 3.....	20
Tabel 6 Source Code soal3.cpp.....	20
Tabel 7 Contoh Kasus Soal 4.....	25
Tabel 8 Source Code soal4.cpp.....	26
Tabel 9 Contoh Kasus Soal 5.....	32
Tabel 10 Source Code soal5.cpp.....	33

SOAL 1

Deskripsi Soal

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

M

$U_1 V_1 W_1$

$U_2 V_2 W_2$

...

$U_M V_M W_M$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

$V_1 V_2 \dots V_N$

$V_1 a_{11} a_{12} \dots a_{1N}$

$V_2 a_{21} a_{22} \dots a_{2N}$

...

VN aN1 aN2 ... aNN

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki self-loop.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Tabel 1 Contoh Kasus Soal 1

Input	Output
5	Adjacency Matrix:
A B C D E	A B C D E
6	A 0 4 2 0 0
A B 4	B 0 0 0 7 0
A C 2	C 0 1 0 0 6
B D 7	D 0 0 3 0 0
C B 1	E 0 0 0 0 0
C E 6	
D C 3	

A. Source Code

Tabel 2 Source Code soal1.cpp

soal1.cpp	
1	#include <bits/stdc++.h>

```

2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      int n, m;
9      cin >> n;
10
11     vector<char> labels;
12     string token;
13
14     while ((int)labels.size() < n && cin >> token) {
15         for (char c : token) {
16             if (labels.size() < n) {
17                 labels.push_back(c);
18             }
19         }
20     }
21
22     cin >> m;
23
24     map<char, int> idx;
25     for (int i = 0; i < n; i++) {
26         idx[labels[i]] = i;
27     }
28
29     vector<vector<int>> adj(n, vector<int>(n, 0));
30
31     for (int i = 0; i < m; i++) {
32         char u, v;
33         int w;
34         cin >> u >> v >> w;
35         adj[idx[u]][idx[v]] = w;
36     }
37
38     cout << "Adjacency Matrix:\n";

```



```
39     cout << " ";
40     for (char c : labels) cout << c;
41     cout << "\n";
42
43     for (int i = 0; i < n; i++) {
44         cout << labels[i] << " ";
45         for (int j = 0; j < n; j++) {
46             cout << adj[i][j];
47         }
48         cout << "\n";
49     }
50
51     return 0;
52 }
```

B. Output Program

```
[fizz@arch7 module-6-graph-and-tree-HerrscherX86]$ ./soal1
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
  ABCDE
A 04200
B 00070
C 01006
D 00300
E 00000
```

Gambar 1 Output Dari Soal 1

C. Pembahasan

Pada baris [1], syntax `#include <bits/stdc++.h>` digunakan untuk mengimpor header file yang berisi semua library standar C++, termasuk vector, map, string, iostream, dan lainnya dalam satu include tunggal (umum digunakan dalam competitive programming).

Pada baris [2], syntax `using namespace std;` digunakan untuk menggunakan namespace std sehingga kita tidak perlu menuliskan `std::` sebelum fungsi-fungsi seperti `cin`, `cout`, `vector`, `map`, dan `string`.

Pada baris [4], syntax `int main()` digunakan untuk mendeklarasikan fungsi utama yang merupakan entry point program dan mengembalikan nilai integer sebagai status program.

Pada baris [5], syntax `ios::sync_with_stdio(false);` digunakan untuk mempercepat input/output dengan menonaktifkan sinkronisasi antara C dan C++ I/O streams.

Pada baris [6], syntax `cin.tie(nullptr);` digunakan untuk memutuskan hubungan antara `cin` dan `cout` sehingga output buffer tidak otomatis di-flush sebelum input, mempercepat operasi I/O.

Pada baris [8], syntax `int n, m;` digunakan untuk mendeklarasikan dua variabel integer `n` dan `m` yang akan menyimpan jumlah node dan jumlah edge dalam graf.

Pada baris [9], syntax `cin >> n;` digunakan untuk membaca input jumlah node dari pengguna dan menyimpannya dalam variabel `n`.

Pada baris [11], syntax `vector<char> labels;` digunakan untuk mendeklarasikan vektor yang akan menyimpan label-label (karakter) dari setiap node dalam graf.

Pada baris [11], syntax `string token;` digunakan untuk mendeklarasikan variabel string yang akan menyimpan input token berupa label-label node.

Pada baris [14], syntax `while ((int)labels.size() < n && cin >> token)` digunakan untuk membaca input token selama jumlah label yang dikumpulkan masih kurang dari `n` dan input berhasil dibaca.

Pada baris [15], syntax `for (char c : token)` digunakan untuk melakukan iterasi setiap karakter dalam token.

Pada baris [16], syntax `if (labels.size() < n)` digunakan untuk mengecek apakah jumlah label masih kurang dari `n`.

Pada baris [17], syntax `labels.push_back(c);` digunakan untuk menambahkan karakter `c` ke dalam vektor `labels` jika jumlahnya masih kurang dari `n`.

Pada baris [21], syntax `cin >> m;` digunakan untuk membaca input jumlah edge (sisi) dalam graf dan menyimpannya dalam variabel `m`.

Pada baris [23], syntax `map<char, int> idx;` digunakan untuk mendeklarasikan map yang memetakan setiap label (karakter) ke indeks numeriknya dalam vektor `labels`.

Pada baris [24-26], syntax `for (int i = 0; i < n; i++) { idx[labels[i]] = i; }` digunakan untuk melakukan iterasi dan membangun map `idx` yang memetakan setiap label ke indeksnya.

Pada baris [29], syntax `vector<vector<int>> adj(n, vector<int>(n, 0));` digunakan untuk mendeklarasikan matriks adjacency berukuran $n \times n$ yang diinisialisasi dengan nilai 0.

Pada baris [31-35], syntax `for (int i = 0; i < m; i++)` digunakan untuk membaca `m` edge dengan format `u v w` (node `u`, node `v`, weight `w`) dan menyimpannya dalam matriks adjacency.

Pada baris [38-48], syntax `cout << "Adjacency Matrix:\n";` dan loop berikutnya digunakan untuk menampilkan matriks adjacency dengan format yang terstruktur, menampilkan label baris, label kolom, dan nilai-nilai adjacency matrix.

Pada baris [46], syntax `return 0;` digunakan untuk menyelesaikan program dan mengembalikan nilai 0 yang mengindikasikan program berjalan tanpa error.

SOAL 2

Deskripsi Soal

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

$A_{11} A_{12} \dots A_{1N}$

$A_{21} A_{22} \dots A_{2N}$

...

$A_{N1} A_{N2} \dots A_{NN}$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

$V_1 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

$V_2 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

...

$V_N \rightarrow \dots$

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol $\rightarrow$.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Tabel 3 Contoh Kasus Soal 2

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4), (C,2) B -> (D,7) C -> (B,1), (E,6) D -> (C,3) E ->

A. Source Code

Tabel 4 Source Code soal2.cpp

soal2.cpp	
1	#include <bits/stdc++.h>
2	using namespace std;
3	
4	int main() {

```

5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      int n;
9      cin >> n;
10
11     vector<char> labels;
12     string token;
13
14     while ((int)labels.size() < n && cin >> token) {
15         for (char c : token) {
16             if (labels.size() < n) {
17                 labels.push_back(c);
18             }
19         }
20     }
21
22     vector<vector<int>> matrix(n, vector<int>(n));
23     for (int i = 0; i < n; i++) {
24         for (int j = 0; j < n; j++) {
25             cin >> matrix[i][j];
26         }
27     }
28
29     cout << "Adjacency List:\n";
30     for (int i = 0; i < n; i++) {
31         cout << labels[i] << "->";
32         bool first = true;
33         bool hasEdge = false;
34         for (int j = 0; j < n; j++) {
35             if (matrix[i][j] > 0) {
36                 if (!first) cout << ",";
37                 cout << "(" << labels[j] << "," << matrix[i][j]
38                 << ")";
39                 first = false;
40                 hasEdge = true;
41             }
42         }
43         if (hasEdge) cout << "\n";
44     }

```

41	}
42	if (!hasEdge) cout << "-";
43	cout << "\n";
44	}
45	
46	return 0;
47	}

B. Output Program

```
[fizz@arch7 module-6-graph-and-tree-HerrscherX86]$ ./soal2
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A->(B,4),(C,2)
B->(D,7)
C->(B,1),(E,6)
D->(C,3)
E->-
```

Gambar 2 Output Dari Soal 2

C. Pembahasan

Pada baris [1], syntax `#include <bits/stdc++.h>` digunakan untuk mengimpor semua library standar C++ dalam satu include tunggal (umum digunakan dalam competitive programming).

Pada baris [2], syntax `using namespace std;` digunakan untuk menggunakan namespace std sehingga tidak perlu menuliskan `std::` sebelum fungsi-fungsi seperti `cin`, `cout`, `vector`, dan `string`.

Pada baris [4], syntax `int main()` digunakan untuk mendeklarasikan fungsi utama yang merupakan entry point program.

Pada baris [5], syntax `ios::sync_with_stdio(false);` digunakan untuk mempercepat input/output dengan menonaktifkan sinkronisasi antara C dan C++ I/O streams.

Pada baris [6], syntax `cin.tie(nullptr);` digunakan untuk memutuskan hubungan antara `cin` dan `cout` sehingga mempercepat operasi I/O.

Pada baris [8], syntax `int n;` digunakan untuk mendeklarasikan variabel integer `n` yang akan menyimpan jumlah node dalam graf.

Pada baris [9], syntax `cin >> n;` digunakan untuk membaca input jumlah node dari pengguna.

Pada baris [11], syntax `vector<char> labels;` digunakan untuk mendeklarasikan vektor yang akan menyimpan label-label (karakter) dari setiap node.

Pada baris [12], syntax `string token;` digunakan untuk mendeklarasikan variabel string yang akan menyimpan input label-label node.

Pada baris [14], syntax `while ((int)labels.size() < n && cin >> token)` digunakan untuk membaca input token selama jumlah label yang dikumpulkan masih kurang dari `n`.

Pada baris [15-19], syntax `for (char c : token) { if (labels.size() < n) { labels.push_back(c); } }` digunakan untuk melakukan iterasi setiap karakter dalam token dan menambahkannya ke vektor `labels` jika jumlahnya masih kurang dari `n`.

Pada baris [21], syntax `vector<vector<int>> matrix(n, vector<int>(n));` digunakan untuk mendeklarasikan matriks adjacency berukuran $n \times n$.

Pada baris [22-26], syntax `for (int i = 0; i < n; i++) { for (int j = 0; j < n; j++) { cin >> matrix[i][j]; } }` digunakan untuk membaca elemen-elemen matriks adjacency dari input dan menyimpannya dalam variabel `matrix`.

Pada baris [29], syntax `cout << "Adjacency List:\n";` digunakan untuk menampilkan header "Adjacency List:" di output.

Pada baris [30], syntax `for (int i = 0; i < n; i++)` digunakan untuk melakukan iterasi setiap baris (node) dalam matriks adjacency.

Pada baris [31], syntax `cout << labels[i] << "->";` digunakan untuk menampilkan label node `i` diikuti dengan panah `"->"`.

Pada baris [32], syntax `bool first = true;` digunakan untuk mendeklarasikan flag boolean yang digunakan untuk mengontrol penambahan koma antar edge.

Pada baris [33], syntax `bool hasEdge = false;` digunakan untuk mendeklarasikan flag boolean yang digunakan untuk mengecek apakah node memiliki edge atau tidak.

Pada baris [34], syntax `for (int j = 0; j < n; j++)` digunakan untuk melakukan iterasi setiap kolom (node tujuan) dalam baris `i`.

Pada baris [35], syntax `if (matrix[i][j] > 0)` digunakan untuk mengecek apakah ada edge dari node `i` ke node `j` (nilai lebih dari 0).

Pada baris [36], syntax `if (!first) cout << ",";` digunakan untuk menampilkan koma jika bukan edge pertama.

Pada baris [37], syntax `cout << "(" << labels[j] << "," << matrix[i][j] << ")";` digunakan untuk menampilkan edge dalam format "(label_node, bobot)".

Pada baris [38], syntax `first = false;` digunakan untuk mengubah flag `first` menjadi false setelah edge pertama ditampilkan.

Pada baris [39], syntax `hasEdge = true;` digunakan untuk menandai bahwa node `i` memiliki edge.

Pada baris [42], syntax `if (!hasEdge) cout << "-";` digunakan untuk menampilkan tanda "-" jika node `i` tidak memiliki edge.

Pada baris [43], syntax `cout << "\n";` digunakan untuk menampilkan newline setelah menampilkan semua edge dari node `i`.

Pada baris [46], syntax `return 0;` digunakan untuk menyelesaikan program dan mengembalikan nilai 0.

SOAL 3

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis

Format Input

Input diberikan dalam format berikut.

R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

S_R S_C

F_R F_C

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Tabel 5 Contoh Kasus Soal 3

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9	16

A. Source Code

Tabel 6 Source Code soal3.cpp

soal3.cpp	
1	#include <bits/stdc++.h>
2	using namespace std;
3	
4	int main() {
5	ios::sync_with_stdio(false);
6	cin.tie(nullptr);
7	
8	int r, c;
9	cin >> r >> c;
10	
11	vector<vector<int>> grid(r, vector<int>(c));
12	for (int i = 0; i < r; i++) {
13	for (int j = 0; j < c; j++) {
14	cin >> grid[i][j];

```

15     }
16 }
17
18 int sr, sc, fr, fc;
19 cin >> sr >> sc;
20 cin >> fr >> fc;
21
22 vector<vector<int>> dist(r, vector<int>(c, -1));
23 queue<pair<int, int>> q;
24
25 dist[sr][sc] = 0;
26 q.push({sr, sc});
27
28 int dr[] = {-1, 1, 0, 0};
29 int dc[] = {0, 0, -1, 1};
30
31 while (!q.empty()) {
32     auto [cr, cc] = q.front(); q.pop();
33
34     if (cr == fr && cc == fc) break;
35
36     for (int i = 0; i < 4; i++) {
37         int nr = cr + dr[i];
38         int nc = cc + dc[i];
39
40         if (nr >= 0 && nr < r && nc >= 0 && nc < c
41             && grid[nr][nc] == 0 && dist[nr][nc] == -1) {
42             dist[nr][nc] = dist[cr][cc] + 1;
43             q.push({nr, nc});
44         }
45     }
46 }
47
48 cout << dist[fr][fc] << "\n";
49
50 return 0;
51 }

```

B. Output Program

```
[fizz@arch7 module-6-graph-and-tree-HerrscherX86]$ ./soal3
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
[fizz@arch7 module-6-graph-and-tree-HerrscherX86]$
```

Gambar 3 Output Dari Soal 3

C. Pembahasan

Pada baris [1], syntax `#include <bits/stdc++.h>` digunakan untuk mengimpor semua library standar C++ dalam satu include tunggal.

Pada baris [2], syntax `using namespace std;` digunakan untuk menggunakan namespace std sehingga tidak perlu menuliskan `std::` sebelum fungsi-fungsi standar.

Pada baris [4], syntax `int main()` digunakan untuk mendeklarasikan fungsi utama program.

Pada baris [5], syntax `ios::sync_with_stdio(false);` digunakan untuk mempercepat input/output dengan menonaktifkan sinkronisasi antara C dan C++ I/O streams.

Pada baris [6], syntax `cin.tie(nullptr);` digunakan untuk memutuskan hubungan antara cin dan cout guna mempercepat operasi I/O.

Pada baris [8-9], syntax `int r, c; cin >> r >> c;` digunakan untuk mendeklarasikan dan membaca jumlah baris dan kolom grid.

Pada baris [11-16], syntax `vector<vector<int>> grid(r, vector<int>(c));` dan loop berikutnya digunakan untuk mendeklarasikan dan membaca elemen-elemen grid 2D yang berisi nilai 0 (jalan) atau 1 (tembok/obstacle).

Pada baris [18-20], syntax `int sr, sc, fr, fc; cin >> sr >> sc; cin >> fr >> fc;` digunakan untuk mendeklarasikan dan membaca posisi awal (sr, sc) dan posisi tujuan (fr, fc).

Pada baris [22], syntax `vector<vector<int>> dist(r, vector<int>(c, -1));` digunakan untuk mendeklarasikan array 2D yang menyimpan jarak dari titik awal ke setiap sel, diinisialisasi dengan -1 (belum dikunjungi).

Pada baris [23], syntax `queue<pair<int, int>> q;` digunakan untuk mendeklarasikan queue yang menyimpan pasangan koordinat (baris, kolom) untuk algoritma BFS.

Pada baris [25-26], syntax `dist[sr][sc] = 0; q.push({sr, sc});` digunakan untuk menginisialisasi jarak titik awal sebagai 0 dan memasukkannya ke dalam queue.

Pada baris [28-29], syntax `int dr[] = {-1, 1, 0, 0}; int dc[] = {0, 0, -1, 1};` digunakan untuk mendeklarasikan array yang berisi perubahan baris dan kolom untuk 4 arah pergerakan (atas, bawah, kiri, kanan).

Pada baris [31], syntax `while (!q.empty())` digunakan untuk melakukan loop BFS selama queue masih memiliki elemen.

Pada baris [32], syntax `auto [cr, cc] = q.front(); q.pop();` digunakan untuk mengambil pasangan koordinat terdepan dari queue dan menghapusnya.

Pada baris [34], syntax `if (cr == fr && cc == fc) break;` digunakan untuk menghentikan BFS jika posisi saat ini adalah posisi tujuan.

Pada baris [36], syntax `for (int i = 0; i < 4; i++)` digunakan untuk melakukan iterasi ke 4 arah pergerakan.

Pada baris [37-38], syntax `int nr = cr + dr[i]; int nc = cc + dc[i];` digunakan untuk menghitung koordinat tetangga berdasarkan arah pergerakan.

Pada baris [40-41], syntax `if (nr >= 0 && nr < r && nc >= 0 && nc < c && grid[nr][nc] == 0 && dist[nr][nc] == -1)` digunakan untuk mengecek apakah tetangga valid (dalam batas grid, bukan obstacle, dan belum dikunjungi).

Pada baris [42-43], syntax `dist[nr][nc] = dist[cr][cc] + 1; q.push({nr, nc});` digunakan untuk menambah jarak tetangga dan memasukkannya ke dalam queue.

Pada baris [48], syntax `cout << dist[fr][fc] << "\n";` digunakan untuk menampilkan jarak terpendek dari titik awal ke titik tujuan.

Pada baris [50], syntax `return 0;` digunakan untuk menyelesaikan program.

SOAL 4

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

$R\ C$

$G_{11}\ G_{12}\ \dots\ G_{1C}$

$G_{21}\ G_{22}\ \dots\ G_{2C}$

...

$G_{R1}\ G_{R2}\ \dots\ G_{RC}$

$S_R\ S_C$

$F_R\ F_C$

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus*Tabel 7 Contoh Kasus Soal 4*

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

A. Source Code

Tabel 8 Source Code soal4.cpp

soal4.cpp	
1	#include <bits/stdc++.h>
2	using namespace std;
3	
4	int r, c, sr, sc, fr, fc;
5	vector<vector<int>> grid;
6	vector<vector<bool>> visited;
7	int pathCount = 0;
8	
9	int dr[] = {-1, 1, 0, 0};
10	int dc[] = {0, 0, -1, 1};
11	
12	void dfs(int cr, int cc) {
13	if (cr == fr && cc == fc) {
14	pathCount++;
15	return;
16	}
17	
18	visited[cr][cc] = true;
19	
20	for (int i = 0; i < 4; i++) {
21	int nr = cr + dr[i];
22	int nc = cc + dc[i];
23	
24	if (nr >= 0 && nr < r && nc >= 0 && nc < c
25	&& grid[nr][nc] == 0 && !visited[nr][nc]) {
26	dfs(nr, nc);
27	}
28	}
29	
30	visited[cr][cc] = false;
31	}
32	
33	int main() {
34	ios::sync_with_stdio(false);

```
35     cin.tie(nullptr);
36
37     cin >> r >> c;
38
39     grid.resize(r, vector<int>(c));
40     for (int i = 0; i < r; i++) {
41         for (int j = 0; j < c; j++) {
42             cin >> grid[i][j];
43         }
44     }
45
46     cin >> sr >> sc;
47     cin >> fr >> fc;
48
49     visited.resize(r, vector<bool>(c, false));
50     dfs(sr, sc);
51
52     cout << pathCount << "\n";
53
54     return 0;
55 }
```

B. Output Program

```
[fizz@arch7 module-6-graph-and-tree-HerrscherX86]$ ./soal4
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
[fizz@arch7 module-6-graph-and-tree-HerrscherX86]$
```

Gambar 4 Output Dari Soal 4

C. Pembahasan

Pada baris [1], syntax `#include <bits/stdc++.h>` digunakan untuk mengimpor semua library standar C++ dalam satu include tunggal.

Pada baris [2], syntax `using namespace std;` digunakan untuk menggunakan namespace std sehingga tidak perlu menuliskan `std::` sebelum fungsi-fungsi standar.

Pada baris [4], syntax `int r, c, sr, sc, fr, fc;` digunakan untuk mendeklarasikan variabel global `r` (baris), `c` (kolom), `sr` (baris awal), `sc` (kolom awal), `fr` (baris tujuan), dan `fc` (kolom tujuan).

Pada baris [5], syntax `vector<vector<int>> grid;` digunakan untuk mendeklarasikan grid 2D secara global yang akan menyimpan nilai 0 (jalan) atau 1 (obstacle).

Pada baris [6], syntax `vector<vector<bool>> visited;` digunakan untuk mendeklarasikan array 2D boolean global yang melacak sel mana yang telah dikunjungi dalam pencarian DFS.

Pada baris [7], syntax `int pathCount = 0;` digunakan untuk mendeklarasikan dan menginisialisasi variabel global yang menghitung jumlah path (jalur) dari titik awal ke titik tujuan.

Pada baris [9-10], syntax `int dr[] = {-1, 1, 0, 0}; int dc[] = {0, 0, -1, 1};` digunakan untuk mendeklarasikan array global yang berisi perubahan baris dan kolom untuk 4 arah pergerakan (atas, bawah, kiri, kanan).

Pada baris [12], syntax `void dfs(int cr, int cc)` digunakan untuk mendeklarasikan fungsi DFS (Depth-First Search) yang menerima koordinat baris dan kolom saat ini.

Pada baris [13], syntax `if (cr == fr && cc == fc)` digunakan untuk mengecek apakah posisi saat ini adalah posisi tujuan.

Pada baris [14], syntax `pathCount++`; digunakan untuk menambah jumlah path jika posisi tujuan tercapai.

Pada baris [15], syntax `return`; digunakan untuk menghentikan eksekusi fungsi DFS ketika posisi tujuan ditemukan.

Pada baris [18], syntax `visited[cr][cc] = true`; digunakan untuk menandai sel saat ini sebagai telah dikunjungi.

Pada baris [20], syntax `for (int i = 0; i < 4; i++)` digunakan untuk melakukan iterasi ke 4 arah pergerakan.

Pada baris [21-22], syntax `int nr = cr + dr[i]; int nc = cc + dc[i]`; digunakan untuk menghitung koordinat tetangga berdasarkan arah pergerakan.

Pada baris [24-25], syntax `if (nr >= 0 && nr < r && nc >= 0 && nc < c && grid[nr][nc] == 0 && !visited[nr][nc])` digunakan untuk mengecek apakah tetangga valid (dalam batas grid, bukan obstacle, dan belum dikunjungi).

Pada baris [26], syntax `dfs(nr, nc)`; digunakan untuk melakukan rekursi DFS ke sel tetangga.

Pada baris [30], syntax `visited[cr][cc] = false`; digunakan untuk menghapus penanda kunjungan pada sel saat ini (backtracking) agar dapat dieksplorasi melalui jalur berbeda.

Pada baris [33], syntax `int main()` digunakan untuk mendeklarasikan fungsi utama program.

Pada baris [34], syntax `ios::sync_with_stdio(false)`; digunakan untuk mempercepat input/output.

Pada baris [35], syntax `cin.tie(nullptr)`; digunakan untuk memutuskan hubungan antara cin dan cout guna mempercepat operasi I/O.

Pada baris [37], syntax `cin >> r >> c`; digunakan untuk membaca jumlah baris dan kolom grid.

Pada baris [39-43], syntax `grid.resize(r, vector<int>(c))`; dan loop berikutnya digunakan untuk mengubah ukuran grid dan membaca elemen-elemen grid 2D.

Pada baris [46-47], syntax `cin >> sr >> sc; cin >> fr >> fc`; digunakan untuk membaca posisi awal dan posisi tujuan.

Pada baris [49], syntax `visited.resize(r, vector<bool>(c, false));` digunakan untuk mengubah ukuran array `visited` dan menginisialisasi semua nilai dengan `false`.

Pada baris [50], syntax `dfs(sr, sc);` digunakan untuk memulai pencarian DFS dari titik awal.

Pada baris [52], syntax `cout << pathCount << "\n";` digunakan untuk menampilkan jumlah path (jalur) dari titik awal ke titik tujuan.

Pada baris [54], syntax `return 0;` digunakan untuk menyelesaikan program.

SOAL 5

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah RedBlack Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N$

Q

T_1

T_2

...

T_Q

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-10000000000 \leq A_i \leq 10000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL

Contoh Kasus

Tabel 9 Contoh Kasus Soal 5

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER	[Preorder] : 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder] : 20 40 30 60 80 70 50
ALL	

A. Source Code

Tabel 10 Source Code soal5.cpp

soal5.cpp	
1	#include <bits/stdc++.h>
2	#include "RedBlackTree.h"
3	using namespace std;
4	
5	void preorder(const RedBlackTree::Node* node, const
	RedBlackTree::Node* nil, vector<int>& res) {
6	if (node->isNil) return;
7	res.push_back(node->key);
8	preorder(node->left, nil, res);
9	preorder(node->right, nil, res);
10	}
11	
12	void inorder(const RedBlackTree::Node* node, const
	RedBlackTree::Node* nil, vector<int>& res) {
13	if (node->isNil) return;
14	inorder(node->left, nil, res);
15	res.push_back(node->key);
16	inorder(node->right, nil, res);
17	}
18	
19	void postorder(const RedBlackTree::Node* node, const
	RedBlackTree::Node* nil, vector<int>& res) {
20	if (node->isNil) return;
21	postorder(node->left, nil, res);
22	postorder(node->right, nil, res);
23	res.push_back(node->key);
24	}
25	
26	void printVector(const vector<int>& v) {
27	for (size_t i = 0; i < v.size(); i++) {
28	if (i > 0) cout << " ";
29	cout << v[i];
30	}

```

31 }
32
33 int main() {
34     ios::sync_with_stdio(false);
35     cin.tie(nullptr);
36
37     int n;
38     cin >> n;
39
40     RedBlackTree tree;
41
42     for (int i = 0; i < n; i++) {
43         int x;
44         cin >> x;
45         if (!tree.contains(x)) {
46             tree.insert(x);
47         }
48     }
49
50     if (tree.empty()) {
51         cout << "Tree kosong. Tidak ada yang bisa ditampilkan.
52         \n";
53         return 0;
54     }
55
56     int q;
57     cin >> q;
58
59     for (int i = 0; i < q; i++) {
60         string type;
61         cin >> type;
62
63         vector<int> pre, in, post;
64
65         if (type == "PREORDER" || type == "ALL") {
66             preorder(tree.root(), tree.nil(), pre);
67         }
68     }
69 }

```

```

67         if (type == "INORDER" || type == "ALL") {
68             inorder(tree.root(), tree.nil(), in);
69         }
70         if (type == "POSTORDER" || type == "ALL") {
71             postorder(tree.root(), tree.nil(), post);
72         }
73
74         if (type == "PREORDER") {
75             cout << "[Preorder]: ";
76             printVector(pre);
77             cout << "\n";
78         } else if (type == "INORDER") {
79             cout << "[Inorder]: ";
80             printVector(in);
81             cout << "\n";
82         } else if (type == "POSTORDER") {
83             cout << "[Postorder]: ";
84             printVector(post);
85             cout << "\n";
86         } else if (type == "ALL") {
87             cout << "[Preorder]: ";
88             printVector(pre);
89             cout << "\n";
90             cout << "[Inorder]: ";
91             printVector(in);
92             cout << "\n";
93             cout << "[Postorder]: ";
94             printVector(post);
95             cout << "\n";
96         }
97     }
98
99     return 0;
100 }

```

B. Output Program

```
[fizz@arch7 module-6-graph-and-tree-HerrscherX86]$ ./soal5
7
50 30 70 20 40 60 80
4
PREORDER
INORDER
POSTORDER
ALL
[Preorder]: 50 30 20 40 70 60 80
[Inorder]: 20 30 40 50 60 70 80
[Postorder]: 20 40 30 60 80 70 50
[Preorder]: 50 30 20 40 70 60 80
[Inorder]: 20 30 40 50 60 70 80
[Postorder]: 20 40 30 60 80 70 50
[fizz@arch7 module-6-graph-and-tree-HerrscherX86]$
```

Gambar 5 Output Dari Soal 5

C. Pembahasan

Pada baris [1], syntax `#include <bits/stdc++.h>` digunakan untuk mengimpor semua library standar C++ dalam satu include tunggal.

Pada baris [2], syntax `#include "RedBlackTree.h"` digunakan untuk mengimpor header file `RedBlackTree.h` yang berisi deklarasi class `RedBlackTree` dan struct `Node`.

Pada baris [3], syntax `using namespace std;` digunakan untuk menggunakan namespace `std` sehingga tidak perlu menuliskan `std::` sebelum fungsi-fungsi standar.

Pada baris [5], syntax `void preorder(const RedBlackTree::Node* node, const RedBlackTree::Node* nil, vector<int>& res)` digunakan untuk mendeklarasikan fungsi `preorder` yang melakukan traversal preorder pada Red-Black Tree (visit node, kiri, kanan) dan menyimpan hasil dalam vektor `res`.

Pada baris [6], syntax `if (node->isNil) return;` digunakan untuk mengecek apakah node adalah nil node (daun), dan menghentikan rekursi jika benar.

Pada baris [7], syntax `res.push_back(node->key);` digunakan untuk menambahkan key dari node saat ini ke vektor hasil.

Pada baris [8-9], syntax `preorder(node->left, nil, res); preorder(node->right, nil, res);` digunakan untuk melakukan rekursi ke anak kiri dan anak kanan.

Pada baris [12], syntax `void inorder(const RedBlackTree::Node* node, const RedBlackTree::Node* nil, vector<int>& res)` digunakan untuk mendeklarasikan fungsi `inorder` yang melakukan traversal inorder pada Red-Black Tree (kiri, visit node, kanan).

Pada baris [13], syntax `if (node->isNil) return;` digunakan untuk mengecek apakah node adalah nil node dan menghentikan rekursi jika benar.

Pada baris [14], syntax `inorder(node->left, nil, res);` digunakan untuk melakukan rekursi ke anak kiri terlebih dahulu.

Pada baris [15], syntax `res.push_back(node->key);` digunakan untuk menambahkan key dari node saat ini ke vektor hasil.

Pada baris [16], syntax `inorder(node->right, nil, res);` digunakan untuk melakukan rekursi ke anak kanan.

Pada baris [19], syntax `void postorder(const RedBlackTree::Node* node, const RedBlackTree::Node* nil, vector<int>& res)` digunakan untuk mendeklarasikan fungsi postorder yang melakukan traversal postorder pada Red-Black Tree (kiri, kanan, visit node).

Pada baris [20], syntax `if (node->isNil) return;` digunakan untuk mengecek apakah node adalah nil node dan menghentikan rekursi jika benar.

Pada baris [21-22], syntax `postorder(node->left, nil, res); postorder(node->right, nil, res);` digunakan untuk melakukan rekursi ke anak kiri dan anak kanan terlebih dahulu.

Pada baris [23], syntax `res.push_back(node->key);` digunakan untuk menambahkan key dari node saat ini ke vektor hasil setelah mengunjungi kedua anak.

Pada baris [26], syntax `void printVector(const vector<int>& v)` digunakan untuk mendeklarasikan fungsi yang menampilkan elemen-elemen vektor.

Pada baris [27-29], syntax `for (size_t i = 0; i < v.size(); i++) { if (i > 0) cout << " "; cout << v[i]; }` digunakan untuk melakukan iterasi elemen vektor dan menampilkannya dengan spasi sebagai pemisah (tanpa spasi sebelum elemen pertama).

Pada baris [33], syntax `int main()` digunakan untuk mendeklarasikan fungsi utama program.

Pada baris [34], syntax `ios::sync_with_stdio(false);` digunakan untuk mempercepat input/output.

Pada baris [35], syntax `cin.tie(nullptr);` digunakan untuk memutuskan hubungan antara cin dan cout guna mempercepat operasi I/O.

Pada baris [37-38], syntax `int n; cin >> n;` digunakan untuk mendeklarasikan dan membaca jumlah elemen yang akan dimasukkan ke dalam Red-Black Tree.

Pada baris [40], syntax `RedBlackTree tree;` digunakan untuk membuat instance dari class `RedBlackTree`.

Pada baris [42-46], syntax `for (int i = 0; i < n; i++) { int x; cin >> x; if (!tree.contains(x)) { tree.insert(x); } }` digunakan untuk membaca `n` elemen dan memasukkannya ke dalam `tree` (hanya jika elemen belum ada di dalam `tree`).

Pada baris [50-52], syntax `if (tree.empty()) { cout << "Tree kosong...\n"; return 0; }` digunakan untuk mengecek apakah `tree` kosong dan menampilkan pesan serta keluar dari program jika benar.

Pada baris [55-56], syntax `int q; cin >> q;` digunakan untuk membaca jumlah query (pertanyaan) traversal yang akan dilakukan.

Pada baris [58], syntax `for (int i = 0; i < q; i++)` digunakan untuk melakukan iterasi setiap query.

Pada baris [59-60], syntax `string type; cin >> type;` digunakan untuk membaca tipe traversal yang diminta (`PREORDER`, `INORDER`, `POSTORDER`, atau `ALL`).

Pada baris [62], syntax `vector<int> pre, in, post;` digunakan untuk mendeklarasikan tiga vektor yang akan menyimpan hasil dari ketiga jenis traversal.

Pada baris [64-65], syntax `if (type == "PREORDER" || type == "ALL") { preorder(tree.root(), tree.nil(), pre); }` digunakan untuk menjalankan fungsi `preorder` jika tipe yang diminta adalah `"PREORDER"` atau `"ALL"`.

Pada baris [67-68], syntax `if (type == "INORDER" || type == "ALL") { inorder(tree.root(), tree.nil(), in); }` digunakan untuk menjalankan fungsi `inorder` jika tipe yang diminta adalah `"INORDER"` atau `"ALL"`.

Pada baris [70-71], syntax `if (type == "POSTORDER" || type == "ALL") { postorder(tree.root(), tree.nil(), post); }` digunakan untuk menjalankan fungsi `postorder` jika tipe yang diminta adalah `"POSTORDER"` atau `"ALL"`.

Pada baris 74-95, syntax `if (type == "PREORDER") { ... } else if (type == "INORDER") { ... } else if (type == "POSTORDER") { ... } else if (type == "ALL") { ... }` digunakan untuk menampilkan hasil traversal sesuai dengan tipe yang diminta.

Pada baris 99, syntax `return 0;` digunakan untuk menyelesaikan program.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-HerrscherX86.git>